Your CI pipeline and your "AI coding standards" are the same rule set.

Most people build them as two things. They drift, and the agent ping-pongs between them. So I built one gate and ran it in three places — in the agent's loop, before the push, and on the PR.

What follows: the thrash that started it, the six-verb contract that fixed it, the guard that stops an agent gaming the metric — and the threshold I had to walk back.

Four commits. Net change: nothing.

01 My local hook flags `oversized-function`. The agent splits the function in two.

|

02 CI flags `high-complexity` — a rule the local hook **never ran**. The agent collapses the branch back inline.

|

03 The local hook flags `oversized-function` again. We are now in a cycle.

└──▶ repeat until I notice

This isn't the agent being dumb. It optimised exactly what it was shown, and it was shown **two different rule sets**.

Every one of these is a placement bug.

SYMPTOM 01 — THRASH

The agent fixes what the hook flagged, CI flags something else, it fixes that and regresses the first.
Cause: three placements, three rule sets.

SYMPTOM 02 — “DONE” MEANT NOTHING

“I've fixed the failing tests and everything passes now.” It hadn't. So I hand-verified everything, which is the work I was trying to delegate.
Cause: no single authority on *done*.

SYMPTOM 03 — THE RULES DECAYED

My standards lived in `CLAUDE.md`. They were followed early in a session and quietly ignored by the end of it.
Cause: prose is advisory. I'd picked the wrong enforcement layer.

One rule set. Three placements. No second definition.

PLACEMENT	FIRES	TALKS TO	BUDGET
in-loop	while the agent edits	the agent, mid-task	< 30s
pre-push	before code leaves my machine	me	< 60s
CI	on the pull request	the permanent record	< 5 min

Same rules, different *audience* and different *latency budget*. A disagreement between two placements is a bug in my setup — never a fact to work around.

Practical consequence: don't write a CI pipeline and then separately write a hooks setup. Write one gate behind a narrow interface and call it from all three.

The narrow interface is six verbs.

`mise run fix`	apply every mechanically safe fix — never a judgement call
`mise run lint`	style + structural smells
`mise run typecheck`	static types
`mise run test`	tests + the coverage floor
`mise run audit`	vulnerabilities, secrets, SAST

`mise run gate`	**all of the above, in order. The oracle.**

> Callers invoke **verbs, never tools**. A skill says `mise run lint`. It never says `eslint` or `phpstan`. That one indirection is why a single skill library serves a Kotlin repo and a React repo.

A verb with nothing to do is still defined, as a no-op that exits 0. A caller must never have to ask whether a verb exists.

The model proposes. The exit code disposes.

DETERMINISTIC — THE ORACLE

Linters, types, tests, scanners. Pinned, reproducible, offline-capable. **These decide pass/fail. Nothing else does.**

PROBABILISTIC — A PROPOSER

A model's output is a *patch*. It's accepted only if the gate then passes from a clean tree.

AND THE THREE MODEL ROLES STAY SEPARATE

`fixer` — failure + coaching in, patch out. Bounded.
`reviewer` — naming, domain drift, “is this the right seam”. **Advisory, never blocking.**
`triager` — flaky vs real, dedupe, route to a ticket.

A model's claim that it fixed something is not evidence. The exit code is.

The reviewer is advisory by construction. The moment a model can block a merge, you start arguing with it instead of shipping.

Ratchet, don't gate.

WHAT EVERYONE TRIES FIRST

Point a good linter at an existing codebase. Everything is red on day one. You turn it off in week two. I've done this more than once.

WHAT ACTUALLY SURVIVES

Record today's violations as an **accepted baseline**, commit it, and allow it to **only ever shrink**. New code is held to the full standard; legacy is held to “no worse”.

`eslint --suppress-all`	bulk-records existing violations; `--prune-suppressions` removes them permanently as you fix them
`habit-snooze`	same ratchet for structural smells
`coverage floor`	set to today's measured number, not an aspiration

Both tools already had this built in. The lesson that cost me the least and saved the most: check whether the ratchet exists before hand-rolling one.

The cheapest fix for `high-complexity` is `// eslint-disable`.

Any ratchet invites the same move: don't fix the code, weaken the rule. Asking nicely in a prompt does not solve this — it's not a discipline problem, it's an incentive.

SO IT'S ENFORCED, NOT REQUESTED

`ruleset-guard`: a PR that touches a ruleset file *and* production source **fails** — unless a human applies the `ruleset-change` label. Weakening a rule gets its own PR, with a name on it.

AND IT'S TIGHTENING-AWARE, BECAUSE THE RISK IS ONE-DIRECTIONAL

`loosening` needs a human. `tightening` and `no semantic change` never do.

A suppression baseline whose count only shrank, a snooze list that only got shorter, a pure line-ending flip — all pass unlabelled. That's what lets `mise run fix` prune the baseline and ship it in the same PR as the fix.

A habit you have to remember isn't a habit.

LAYER	MECHANISM	CHARACTER
reflex	hooks	involuntary — fires unbidden
practice	skills	invoked, procedural, has judgement
standing context	AGENTS.md	ambient shared vocabulary

Picking the wrong layer is why most “AI coding standards” documents end up ignored. My standards were prose, so they decayed. Now the mechanical half is a hook and the prose only carries the judgement.

PLACEMENT FOLLOWS COST — THIS PART IS NOT OPTIONAL

A single-file `prettier` run is sub-second → fire it on **every edit**.
The smell sensor costs ~25s cold, ~6s warm → fire it **once, before the agent declares done**.
Gradle's JVM start-up is seconds → the Java formatter is **once per turn**, not per edit.

Get the cadence wrong and the right check becomes a tax you disable.

I shipped a threshold without its reason.

THE OVER-CORRECTION

The smell gate flagged long functions, so the agent made functions short. Every coherent step became a two-line helper. Technically green, measurably worse to read — I'd traded one real problem for a pile of incohesive fragments.

THE FIX WASN'T A LOOSER NUMBER

It was writing down what the number is *for*: **cohesion is the goal; length and complexity are the honest signals that you don't have it.** Don't chase the number. A straight-line sequence of steps at one level is fine as one function — not every step wants to be a helper.

WHAT CHANGED

file-length default 200 → 300 lines · the standard now carries its rationale and a litmus test (*a helper needing five parameters means the seam is wrong*) · plus an allow-marker for what the gate can't judge

A threshold with no stated rationale will be optimised against — by an agent, and eventually by me.

Five that cost real time.

01 · A REQUIRED CHECK THAT'S PATH-FILTERED OUT NEVER ARRIVES

A docs-only PR shouldn't run the TS gate — but a *required* check filtered out reports `Expected` forever and the PR can't merge. Require a tiny always-running aggregate job that `needs:` the real ones. Skipped ≠ failed.

02 · PIN THE TOOL, NOT JUST THE ACTION

I pinned the setup action by SHA. It still resolved the tool itself from a version endpoint that advertised a release before its assets existed. Both jobs died in under ten seconds.

03 · REQUIRED-CHECK NAMES MATCH THE RENDERED STRING, EXACTLY

Renaming a job to add “(report-only)” silently detached its branch protection. PRs hung waiting on a check that was never coming.

04 · THE TEST ENVIRONMENT WAS THE WHOLE TEST BILL

Defaulting to a node environment and opting into jsdom per file took a suite from **20.0s to 2.6s**. In-loop feedback lives or dies on this.

05 · A BARE EXCLUSION PATTERN MATCHED NOTHING — AND SCANNED NOTHING

The smell config looked correct and was silently a no-op for days. A check that can pass vacuously is worse than no check at all.

Route every lesson to the layer that enforces it.

AT THE END OF A SESSION, IN PRIORITY ORDER

1. a deterministic check — makes the mistake *impossible*
2. `AGENTS.md` — standing context
3. a new skill — a reusable procedure
4. a memory note — last resort

That ordering *is* the whole thesis as a procedure: prefer the placement that makes a mistake impossible over the one that reminds you not to make it. A one-off fix today becomes a rule the next session can't skip.

Two repos, one seam. `foundry` is the CI half, consumed by repositories. `cmaintz-skills` is the agent half, consumed as a plugin. The verb contract is copied verbatim into both — if those copies ever need to differ, the split was wrong.

Proven on TypeScript and Java (a Spring Boot + Angular monorepo). Python, PHP, Kotlin and .NET have templates. MIT, and the links are in the post.
